DBMS Level 3 Project Report

https://github.com/vxs01/Medgraph

1. Project Title

MedGraph: Semantic Clinical Records and Diagnostics Tracker

2. Student SRNs and Names

Sakaray Vishal, PES1UG24AM241
Samarth Ambekar, PES1UG24AM243
AIML 5E

3. Abstract

When a doctor meets an unusual set of symptoms, the most useful thing is often to look at similar patients from the past. Most hospital record systems only offer keyword search, so an old note turns up only if it happens to use the same words. A note that says "progressive memory loss and personality change" will not appear for a search on "becoming forgetful and withdrawn", even though both describe the same problem.

MedGraph is our attempt to fix this. The structured data, which means patients, wards, admissions, medications, prescriptions and staff accounts, is kept in PostgreSQL with constraints, triggers and transactions so that it stays consistent. The free text of each clinical note is also turned into a vector embedding and stored next to the note using the pgvector extension. A doctor can describe a case in ordinary words and the system returns past cases with a similar meaning. The search can be narrowed by age, gender, date or whether the patient is currently admitted, and it can also be combined with normal full text search.

The web application is built with Next.js and has two roles, doctor and admin. It has a dashboard, a patient history page and an audit log. It goes beyond basic create, read, update and delete in a few places: moving a patient between wards is done by a stored procedure, prescriptions stay correct when two doctors work at the same time, and every change is recorded in the audit log together with the user who made it.

4. Technology Stack

Frontend: Next.js 14 (App Router) with React 18.

Backend and API: Next.js route handlers running on Node.js.

Database: PostgreSQL 16 with the pgvector extension, so the relational data and the vectors live in one database.

Embeddings: the all-MiniLM-L6-v2 model (384 dimensions), run locally through the @huggingface/transformers package. No external API is needed.

Authentication: bcryptjs for password hashing, and a signed JWT (jose) kept in an httpOnly cookie.

Database driver: node-postgres (pg), with parameterised queries only.

Other tools: Docker for running PostgreSQL with pgvector, and Git with GitHub for version control.

We chose PostgreSQL with pgvector instead of a separate vector database such as Qdrant or Chroma because the vector sits in the same row as the note. One SQL statement can then combine similarity search with filters on patient data, the note is removed together with its patient by the same cascade, and there is no second system to keep in sync.

5. SQL - Database Commands

All SQL is kept in the sql folder of the repository and is run in order by npm run db:setup. The commands below are the main ones, shortened a little to make them easier to read. Constraint names and table comments are left out here but are in the repository.

5.1 Creating the tables

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE users (
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name VARCHAR(100) NOT NULL CHECK (length(trim(full_name)) > 0),
  email VARCHAR(100) NOT NULL UNIQUE CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(10) NOT NULL CHECK (role IN ('DOCTOR','ADMIN')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE patients (
  patient_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name VARCHAR(50) NOT NULL CHECK (length(trim(first_name)) > 0),
  last_name VARCHAR(50) NOT NULL CHECK (length(trim(last_name)) > 0),
  dob DATE NOT NULL CHECK (dob > DATE '1900-01-01'),
  gender CHAR(1) NOT NULL CHECK (gender IN ('M','F','O')),
  blood_group VARCHAR(3) NOT NULL
      CHECK (blood_group IN ('A+','A-','B+','B-','AB+','AB-','O+','O-')),
  phone_number VARCHAR(20) NOT NULL CHECK (phone_number ~ '^[0-9+ -]{7,20}$'),
  emergency_contact VARCHAR(20) NOT NULL
      CHECK (emergency_contact ~ '^[0-9+ -]{7,20}$'),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE wards (
  ward_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ward_name VARCHAR(50) NOT NULL UNIQUE,
  total_capacity INT NOT NULL CHECK (total_capacity > 0),
  occupied_beds INT NOT NULL DEFAULT 0 CHECK (occupied_beds >= 0),
  CHECK (occupied_beds <= total_capacity)
);

CREATE TABLE medications (
  med_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  med_name VARCHAR(100) NOT NULL UNIQUE,
  stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
  unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0)
);

CREATE TABLE admissions (
  admission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,
  ward_id UUID REFERENCES wards(ward_id) ON DELETE SET NULL,
  doctor_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
  admission_date TIMESTAMPTZ NOT NULL DEFAULT now(),
  discharge_date TIMESTAMPTZ,
```

```
  status VARCHAR(10) NOT NULL DEFAULT 'ADMITTED'
      CHECK (status IN ('ADMITTED','DISCHARGED')),
  CHECK (discharge_date IS NULL OR discharge_date >= admission_date),
  CHECK ((status = 'ADMITTED' AND discharge_date IS NULL) OR
         (status = 'DISCHARGED' AND discharge_date IS NOT NULL))
);

CREATE TABLE clinical_notes (
  note_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,
  doctor_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
  chief_complaint TEXT NOT NULL CHECK (length(trim(chief_complaint)) > 0),
  observation_text TEXT NOT NULL CHECK (length(trim(observation_text)) > 0),
  embedding vector(384),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE prescriptions (
  prescription_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,
  doctor_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
  med_id UUID NOT NULL REFERENCES medications(med_id) ON DELETE RESTRICT,
  dosage VARCHAR(50) NOT NULL CHECK (length(trim(dosage)) > 0),
  quantity INT NOT NULL CHECK (quantity > 0),
  prescribed_date TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE audit_logs (
  log_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
  action_type VARCHAR(50) NOT NULL,
  description TEXT NOT NULL,
  logged_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);
```

5.2 Indexes

```
CREATE INDEX idx_admissions_patient ON admissions(patient_id);
CREATE INDEX idx_notes_patient ON clinical_notes(patient_id);
CREATE INDEX idx_rx_med ON prescriptions(med_id);
CREATE INDEX idx_patients_name ON patients(lower(last_name), lower(first_name));
CREATE INDEX idx_admissions_active_ward ON admissions(ward_id)
    WHERE status = 'ADMITTED';
CREATE UNIQUE INDEX uq_one_active_admission ON admissions(patient_id)
    WHERE status = 'ADMITTED';
CREATE INDEX idx_notes_embedding_hnsw ON clinical_notes
    USING hnsw (embedding vector_cosine_ops);
```

5.3 Functions, procedures and triggers

Business rule errors use their own error codes (MG001 for a full ward, MG002 for a user who is not a doctor, MG003 for not found, MG004 for not enough stock) so the application can show a clear message. The function below admits a patient and locks the ward row, so two people cannot take the last bed at the same time.

```
CREATE FUNCTION admit_patient(p_patient UUID, p_ward UUID, p_doctor UUID) RETURNS UUID
    AS $$
DECLARE v_cap INT; v_occ INT; v_id UUID;
BEGIN
  SELECT total_capacity, occupied_beds INTO v_cap, v_occ FROM wards
      WHERE ward_id = p_ward FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Ward not found' USING ERRCODE = 'MG003';
  END IF;
  IF v_occ >= v_cap THEN
    RAISE EXCEPTION 'That ward is full' USING ERRCODE = 'MG001';
  END IF;
  INSERT INTO admissions(patient_id, ward_id, doctor_id) VALUES (p_patient, p_ward,
      p_doctor)
    RETURNING admission_id INTO v_id;
  RETURN v_id;
END $$ LANGUAGE plpgsql;
```

Transferring a patient is a stored procedure. It locks both wards in a fixed order, which stops two opposite transfers from blocking each other forever.

```
CREATE PROCEDURE transfer_patient(p_admission UUID, p_new_ward UUID) LANGUAGE plpgsql
    AS $$
DECLARE v_old UUID; v_cap INT; v_occ INT;
BEGIN
  SELECT ward_id INTO v_old FROM admissions
    WHERE admission_id = p_admission AND status = 'ADMITTED' FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'No active admission with that id' USING ERRCODE = 'MG003';
  END IF;
  PERFORM ward_id FROM wards WHERE ward_id IN (v_old, p_new_ward) ORDER BY ward_id
      FOR UPDATE;
  SELECT total_capacity, occupied_beds INTO v_cap, v_occ FROM wards
      WHERE ward_id = p_new_ward;
  IF v_occ >= v_cap THEN
    RAISE EXCEPTION 'That ward is full' USING ERRCODE = 'MG001';
  END IF;
  UPDATE admissions SET ward_id = p_new_ward WHERE admission_id = p_admission;
END $$;

CALL transfer_patient('<admission_id>', '<ward_id>');
```

A trigger takes the units out of stock when a prescription is written and refuses if there are not enough. Other triggers keep the ward bed counts in step with admissions, put stock back when a prescription is deleted, check that a doctor is really a doctor, and write the audit log.

```
CREATE FUNCTION fn_decrement_stock() RETURNS trigger AS $$
DECLARE v_stock INT;
BEGIN
  SELECT stock_quantity INTO v_stock FROM medications WHERE med_id = NEW.med_id
      FOR UPDATE;
  IF v_stock IS NOT NULL AND v_stock < NEW.quantity THEN
    RAISE EXCEPTION 'Only % units in stock', v_stock USING ERRCODE = 'MG004';
  END IF;
  UPDATE medications SET stock_quantity = stock_quantity - NEW.quantity
      WHERE med_id = NEW.med_id;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_rx_stock BEFORE INSERT ON prescriptions
  FOR EACH ROW EXECUTE FUNCTION fn_decrement_stock();
```

5.4 Views

```
CREATE VIEW v_ward_occupancy AS
SELECT ward_id, ward_name, total_capacity, occupied_beds,
  total_capacity - occupied_beds AS available_beds,
  ROUND(100.0 * occupied_beds / total_capacity, 1) AS occupancy_pct
FROM wards;

CREATE VIEW v_active_admissions AS
SELECT a.admission_id, a.patient_id, p.first_name || ' ' || p.last_name
    AS patient_name,
  w.ward_name, u.full_name AS doctor_name, a.admission_date
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id
LEFT JOIN wards w ON w.ward_id = a.ward_id
LEFT JOIN users u ON u.user_id = a.doctor_id
WHERE a.status = 'ADMITTED';

-- one timeline per patient, built from four tables with UNION ALL
CREATE VIEW v_patient_timeline AS
SELECT patient_id, admission_date AS event_time, 'ADMISSION' AS event_type
    FROM admissions
UNION ALL
SELECT patient_id, discharge_date, 'DISCHARGE' FROM admissions
    WHERE status = 'DISCHARGED'
UNION ALL
SELECT patient_id, created_at, 'NOTE' FROM clinical_notes
UNION ALL
SELECT patient_id, prescribed_date, 'PRESCRIPTION' FROM prescriptions;
```

The view above is shortened. The version in the repository also carries a summary text and the doctor for each event.

5.5 ALTER and DROP

```
-- generated column and index for full text search
ALTER TABLE clinical_notes ADD COLUMN search_tsv tsvector GENERATED ALWAYS AS (
  setweight(to_tsvector('english', coalesce(chief_complaint, '')), 'A') ||
  setweight(to_tsvector('english', coalesce(observation_text, '')), 'B')) STORED;
CREATE INDEX idx_notes_fts ON clinical_notes USING gin (search_tsv);

-- a rule added after the table was created
ALTER TABLE prescriptions ADD CONSTRAINT chk_rx_quantity_max CHECK (quantity <= 1000);

-- add, rename and drop a column; create and drop an index and a table
ALTER TABLE patients ADD COLUMN email VARCHAR(100);
ALTER TABLE patients RENAME COLUMN email TO contact_email;
ALTER TABLE patients DROP COLUMN contact_email;
CREATE TABLE tmp_demo (id INT PRIMARY KEY, note TEXT);
DROP TABLE tmp_demo;
```

5.6 Queries

Join with GROUP BY, HAVING and ORDER BY:

```
SELECT m.med_name, SUM(rx.quantity) AS units, SUM(rx.quantity * m.unit_price)
    AS revenue
FROM prescriptions rx JOIN medications m ON m.med_id = rx.med_id
GROUP BY m.med_name HAVING SUM(rx.quantity) > 10 ORDER BY revenue DESC;
```

Left join used to find patients who were never admitted:

```
SELECT p.first_name, p.last_name
FROM patients p LEFT JOIN admissions a ON a.patient_id = p.patient_id
WHERE a.admission_id IS NULL;
```

Correlated subquery for each patient's latest note:

```
SELECT p.first_name, n.chief_complaint, n.created_at
FROM patients p JOIN clinical_notes n ON n.patient_id = p.patient_id
WHERE n.created_at = (SELECT MAX(n2.created_at) FROM clinical_notes n2
    WHERE n2.patient_id = p.patient_id);
```

Set operations: patients who have notes and prescriptions, and patients who have notes but no prescription:

```
SELECT patient_id FROM clinical_notes INTERSECT SELECT patient_id FROM prescriptions;
SELECT patient_id FROM clinical_notes EXCEPT SELECT patient_id FROM prescriptions;
```

Recursive CTE that builds the last seven days so that days with no admissions still show as zero (used on the dashboard):

```
WITH RECURSIVE days(day) AS (
  SELECT current_date - 6
  UNION ALL
  SELECT day + 1 FROM days WHERE day < current_date
)
SELECT d.day, COUNT(a.admission_id) AS admissions
FROM days d LEFT JOIN admissions a ON a.admission_date::date = d.day
GROUP BY d.day ORDER BY d.day;
```

Window functions for ranking doctors and for a running count of notes:

```
SELECT full_name, notes_written, RANK() OVER (ORDER BY notes_written DESC) AS rank
    FROM v_doctor_workload;

SELECT u.full_name, n.created_at::date AS day,
  COUNT(*) OVER (PARTITION BY u.user_id ORDER BY n.created_at) AS running_total
FROM clinical_notes n JOIN users u ON u.user_id = n.doctor_id;
```

Division, written as a double NOT EXISTS: doctors who have admitted a patient to every ward.

```
SELECT u.full_name FROM users u
WHERE u.role = 'DOCTOR' AND NOT EXISTS (
  SELECT 1 FROM wards w WHERE NOT EXISTS (
    SELECT 1 FROM admissions a WHERE a.doctor_id = u.user_id
        AND a.ward_id = w.ward_id));
```

Full text search with ranking and a highlighted excerpt:

```
SELECT chief_complaint, ts_rank(search_tsv, q) AS rank,
  ts_headline('english', observation_text, q, 'StartSel=*, StopSel=*') AS excerpt
FROM clinical_notes, websearch_to_tsquery('english', 'chest pain') AS q
WHERE search_tsv @@ q ORDER BY rank DESC;
```

5.7 Insert, update and delete

```
INSERT INTO medications(med_name, stock_quantity, unit_price) VALUES ('Demo A', 10,
    1.00), ('Demo B', 20, 2.00);

-- add a new medication, or add to the stock if it already exists
INSERT INTO medications(med_name, stock_quantity, unit_price) VALUES ('Demo A', 5,
    1.00)
ON CONFLICT (med_name) DO UPDATE
    SET stock_quantity = medications.stock_quantity + EXCLUDED.stock_quantity;

UPDATE medications SET unit_price = ROUND(unit_price * 1.05, 2)
    WHERE med_name LIKE 'Demo%';

-- deleting a patient also deletes their admissions, notes and prescriptions (ON
-- DELETE CASCADE)
DELETE FROM patients WHERE patient_id = '<patient_id>';

-- a transaction: nothing is saved unless every step works
BEGIN;
  SELECT unit_price FROM medications WHERE med_id = '<med_id>' FOR UPDATE;
  INSERT INTO prescriptions(patient_id, doctor_id, med_id, dosage, quantity)
      VALUES ('<p>', '<d>', '<m>', '500mg twice daily', 10);
COMMIT;
```

6. NoSQL/Vector Database Commands

We used the SQL plus vector database option. The vector side is handled by pgvector inside PostgreSQL. We did not add a separate NoSQL database, because the clinical data has a fixed shape and relies on foreign keys and constraints, which a document store would not give us.

6.1 How the embeddings work

What is converted: the chief complaint and the observation text of every clinical note are joined into one piece of text and converted into a vector. The text a doctor types into the search box is converted in the same way.

Model: all-MiniLM-L6-v2, which produces 384 numbers for each text. It runs inside the application through @huggingface/transformers, using mean pooling and normalised output. It is a general purpose model, not one trained on medical text.

Storage: the vector is stored in the embedding column of the clinical_notes table, which has the type vector(384). An HNSW index on that column makes nearest neighbour search fast.

Similarity: cosine similarity. In pgvector the operator <=> gives cosine distance, so similarity is 1 minus the distance.

Retrieval: the search text is embedded, then the notes are ordered by distance to that vector and the closest ones are returned together with the patient details. Ordinary filters such as age, gender and admission status go into the same WHERE clause.

6.2 Commands

```
CREATE EXTENSION IF NOT EXISTS vector;

-- column and index (from the table definition and index script)
embedding vector(384)
CREATE INDEX idx_notes_embedding_hnsw ON clinical_notes
    USING hnsw (embedding vector_cosine_ops);
```

Creating an embedding in the application (lib/embed.mjs):

```
const { pipeline } = await import('@huggingface/transformers');
const extractor = await pipeline('feature-extraction', 'Xenova/all-MiniLM-L6-v2');
const out = await extractor(text, { pooling: 'mean', normalize: true });
const vector = '[' + Array.from(out.data).join(',') + ']';
```

Saving a note together with its embedding:

```
INSERT INTO clinical_notes(patient_id, doctor_id, chief_complaint, observation_text,
    embedding)
VALUES ($1, $2, $3, $4, $5::vector);
```

Searching by meaning with cosine similarity:

```
SELECT n.chief_complaint, p.first_name || ' ' || p.last_name AS patient,
  ROUND((1 - (n.embedding <=> $1::vector))::numeric, 3) AS score
FROM clinical_notes n JOIN patients p ON p.patient_id = n.patient_id
WHERE n.embedding IS NOT NULL
ORDER BY n.embedding <=> $1::vector
LIMIT 5;
```

The same search limited to patients over 50 who are currently admitted:

```
SELECT n.chief_complaint, patient_age(p.dob) AS age,
    ROUND((1 - (n.embedding <=> $1::vector))::numeric, 3) AS score
FROM clinical_notes n JOIN patients p ON p.patient_id = n.patient_id
WHERE n.embedding IS NOT NULL AND patient_age(p.dob) > 50
  AND EXISTS (SELECT 1 FROM admissions a WHERE a.patient_id = p.patient_id
      AND a.status = 'ADMITTED')
ORDER BY n.embedding <=> $1::vector LIMIT 5;
```

More like this, which uses a note that is already stored, so no model call is needed:

```
SELECT n2.chief_complaint, ROUND((1 - (n2.embedding <=> n1.embedding))::numeric, 3)
    AS similarity
FROM clinical_notes n1 JOIN clinical_notes n2 ON n2.note_id <> n1.note_id
WHERE n1.note_id = $1
ORDER BY n2.embedding <=> n1.embedding LIMIT 5;
```

Hybrid search, which ranks by meaning and by keywords separately and then adds 1/(60 + rank) from each list (Reciprocal Rank Fusion). This is a simplified version of the query in lib/search.js:

```
WITH sem AS (
  SELECT note_id, ROW_NUMBER() OVER (ORDER BY embedding <=> $1::vector) AS r
  FROM clinical_notes WHERE embedding IS NOT NULL
  ORDER BY embedding <=> $1::vector LIMIT 20),
kw AS (
  SELECT note_id, ROW_NUMBER() OVER (ORDER BY ts_rank(search_tsv, q) DESC) AS r
  FROM clinical_notes, websearch_to_tsquery('english', $2) AS q
  WHERE search_tsv @@ q ORDER BY ts_rank(search_tsv, q) DESC LIMIT 20)
SELECT n.chief_complaint, COALESCE(1.0 / (60 + sem.r),
    0) + COALESCE(1.0 / (60 + kw.r), 0) AS score
FROM sem FULL OUTER JOIN kw ON kw.note_id = sem.note_id
JOIN clinical_notes n ON n.note_id = COALESCE(sem.note_id, kw.note_id)
ORDER BY score DESC LIMIT 5;
```

7. Next.js - Project Tree Structure

Pages are in the app folder, the API route handlers are under app/api, and the SQL scripts are in the sql folder.

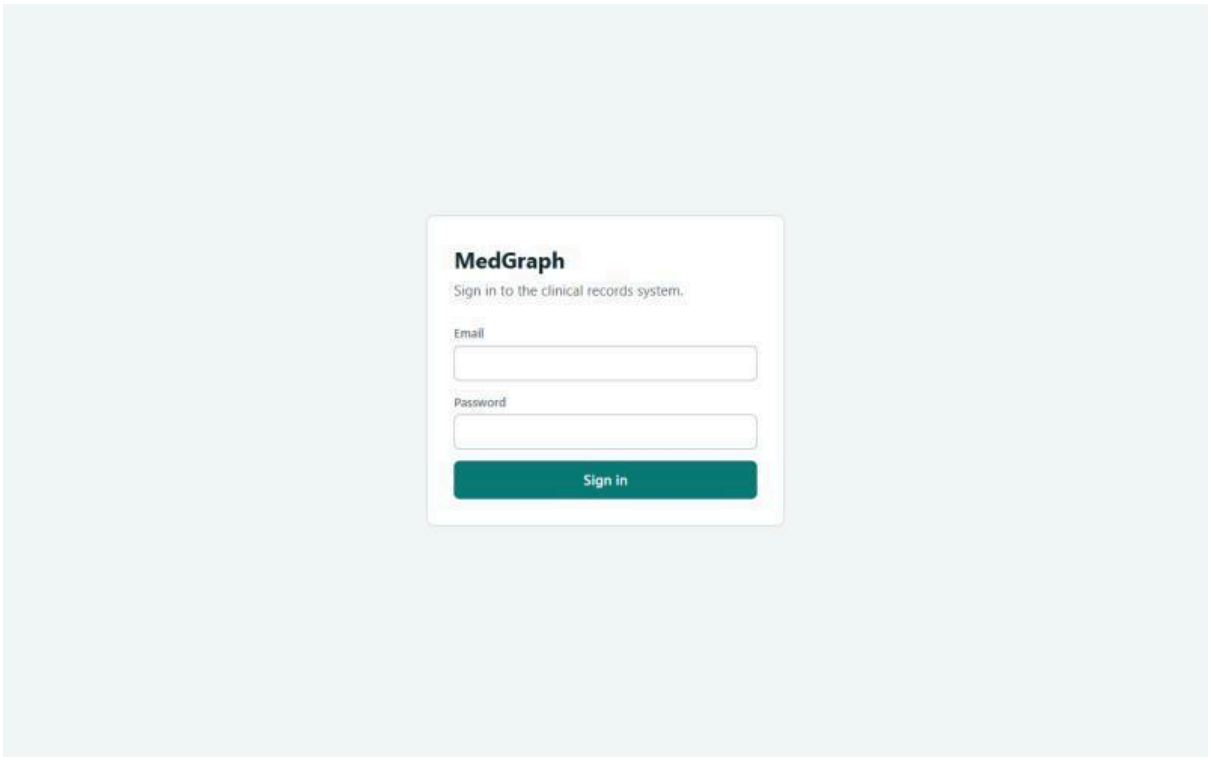
```
medgraph
  app
    layout.js
    page.js (dashboard)
    globals.css
    login/page.js
    patients/page.js
    patients/[id]/page.js (patient history)
    admissions/page.js
    notes/page.js
    search/page.js (similar case search)
    prescriptions/page.js
    medications/page.js
    audit/page.js
    api
      auth/login/route.js
      auth/logout/route.js
      patients/route.js
      patients/[id]/route.js
      admissions/route.js
      admissions/[id]/route.js
      notes/route.js
      notes/[id]/route.js
      search/route.js
      prescriptions/route.js
      prescriptions/[id]/route.js
      medications/route.js
      wards/route.js
      doctors/route.js
      dashboard/route.js
      audit/route.js
  components
    Shell.js
    Msg.js
  lib
    db.js (connection pool and transactions)
    auth.js
    errors.js
    search.js (builds the search queries)
    embed.mjs (creates embeddings)
```

```
    hooks.js
    client.js
    validate.js
  sql
    01_schema.sql
    02_indexes.sql
    03_functions_triggers.sql
    04_views.sql
    05_alter_drop_demo.sql
    06_queries.sql
    07_dml_examples.sql
    08_transaction_demo.sql
    09_explain.sql
  scripts
    setup-db.mjs
    seed.mjs
    test-db.mjs
    concurrency-demo.mjs
    explain-demo.mjs
  docs
  middleware.js
  next.config.mjs
  package.json
  docker-compose.yml
  .env.example
  README.md
```

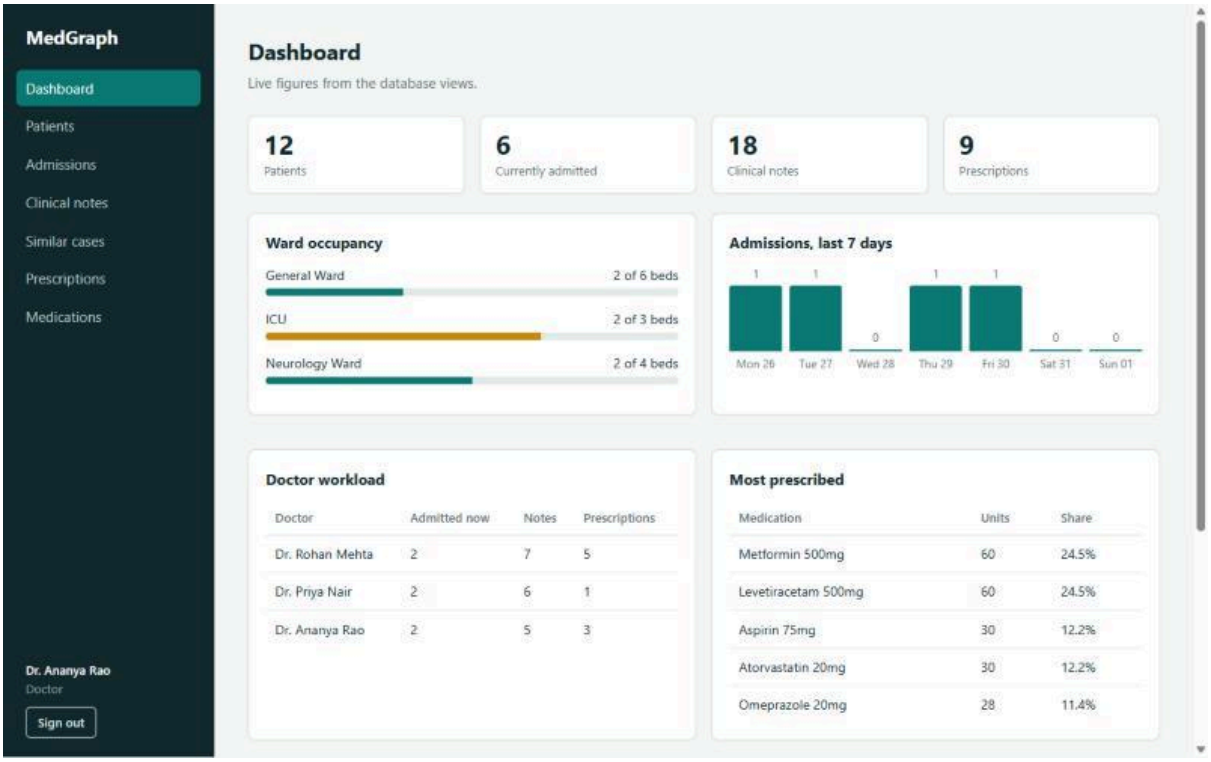
8. Sample Screenshots of Application Code and Demo

The screenshots of the running application and of the code are given below.

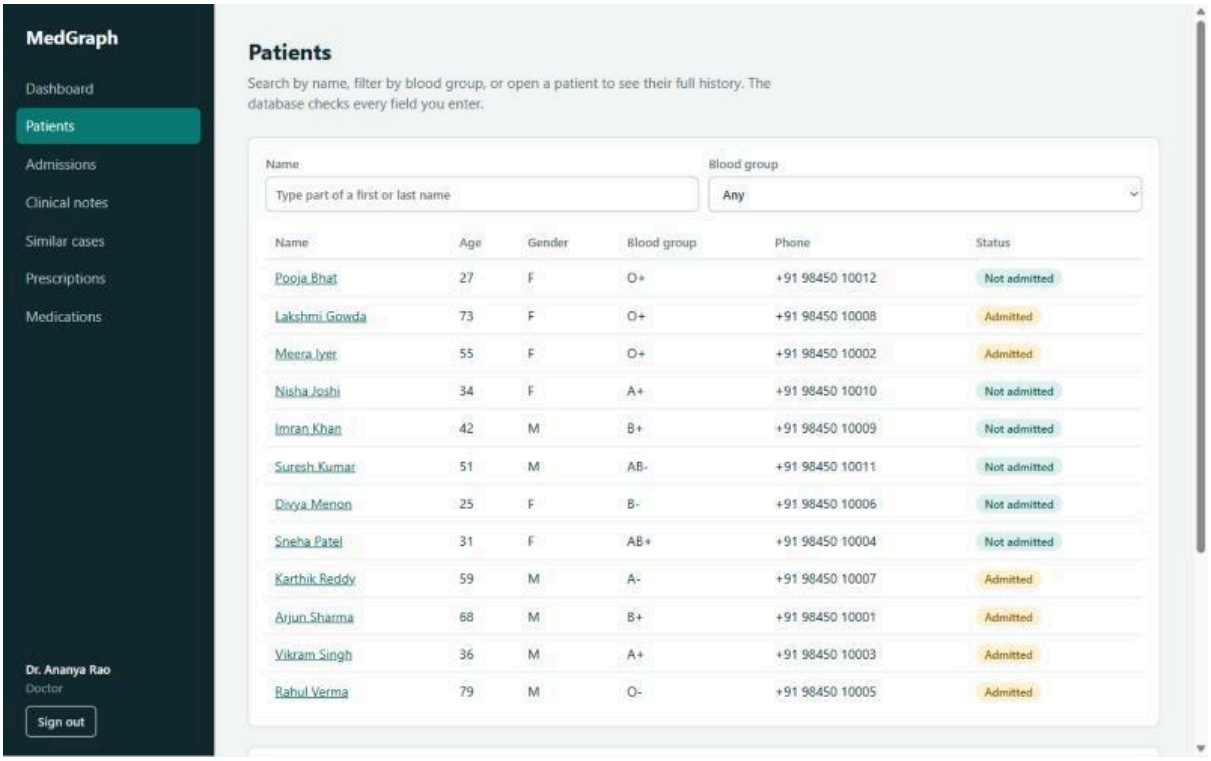
Screenshot 1: Login page



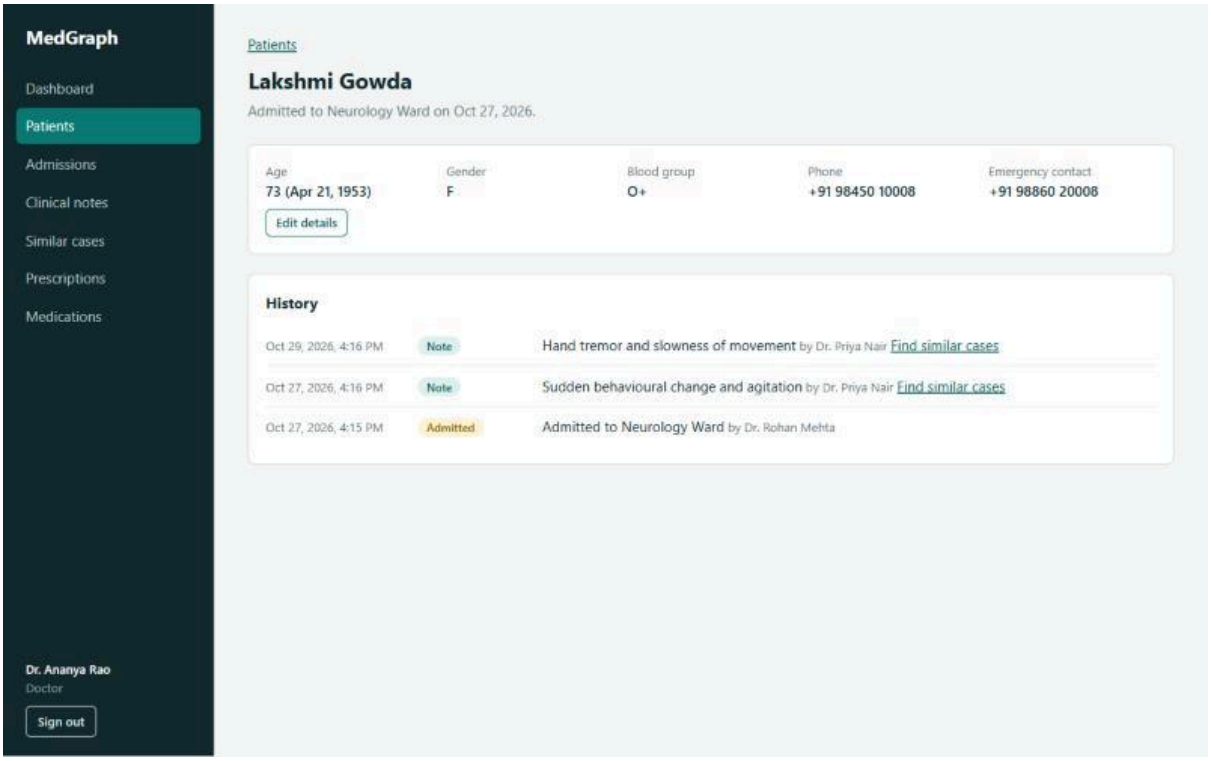
Screenshot 2: Dashboard showing ward occupancy, admissions per day and doctor workload



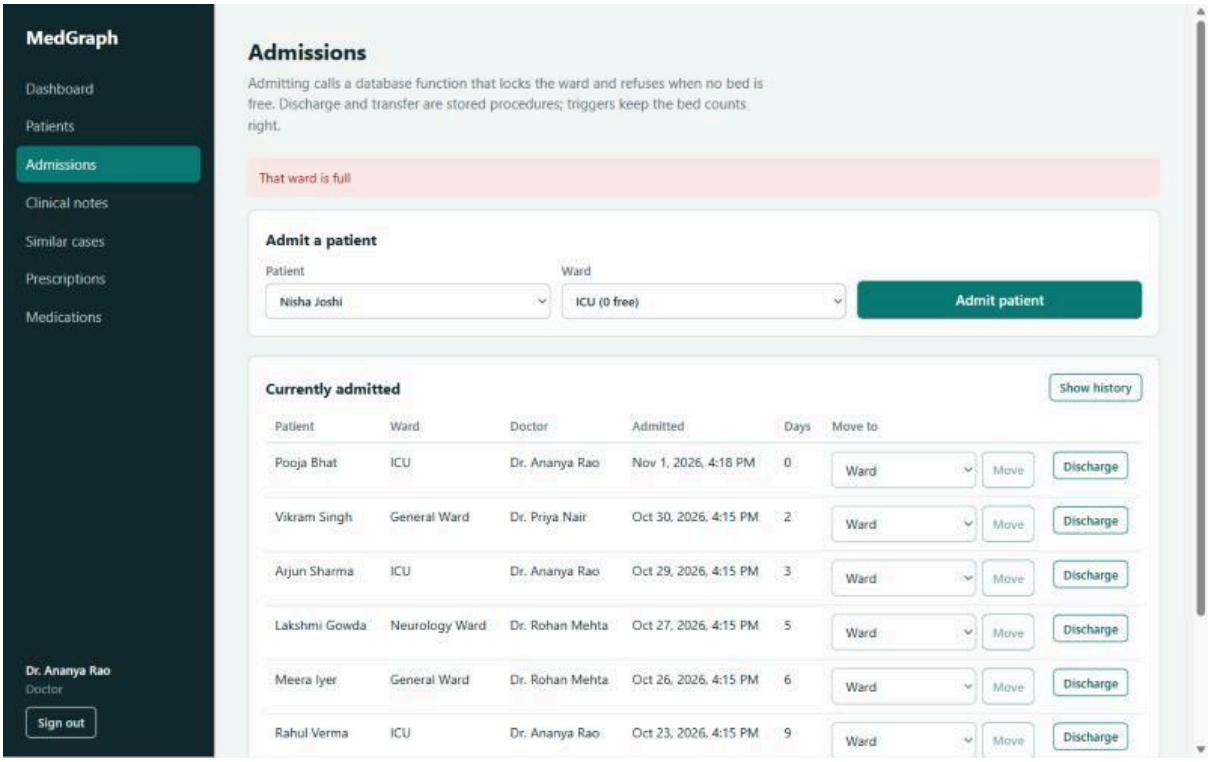
Screenshot 3: Patients list with search and blood group filter



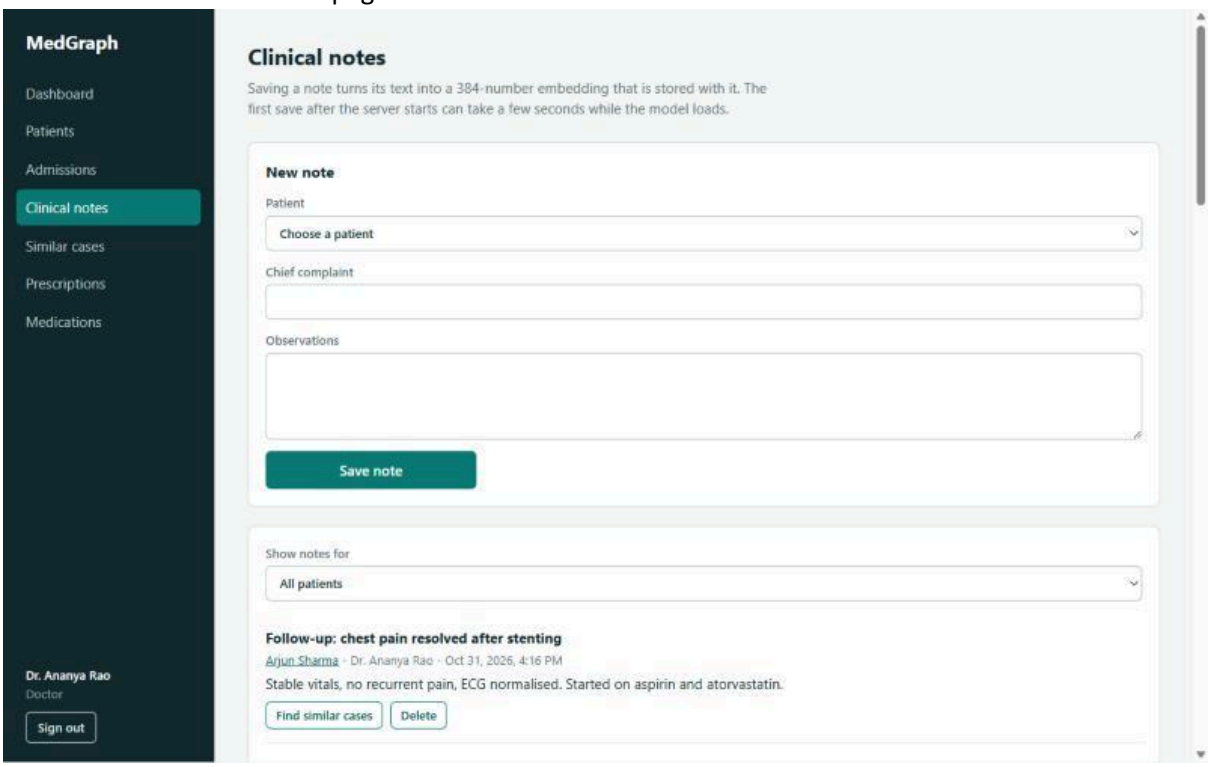
Screenshot 4: Patient history page



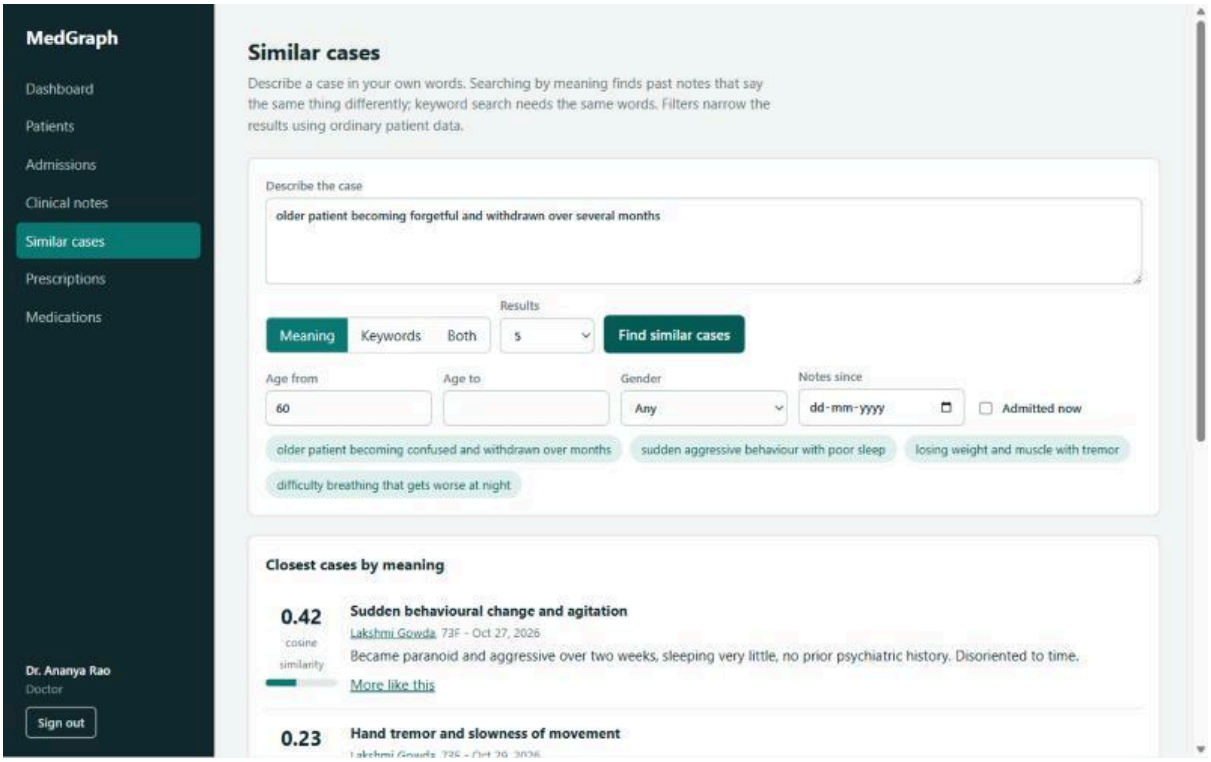
Screenshot 5: Admissions page, including the message shown when a ward is full

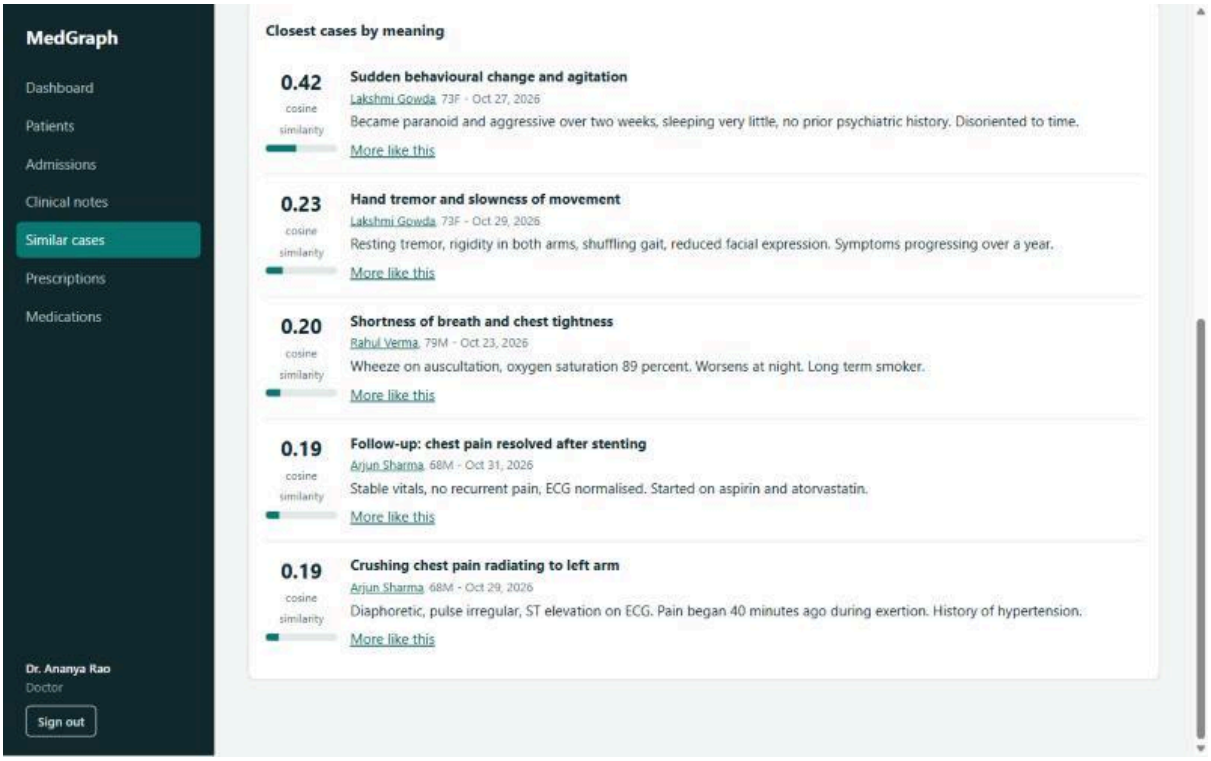


Screenshot 6: Clinical notes page

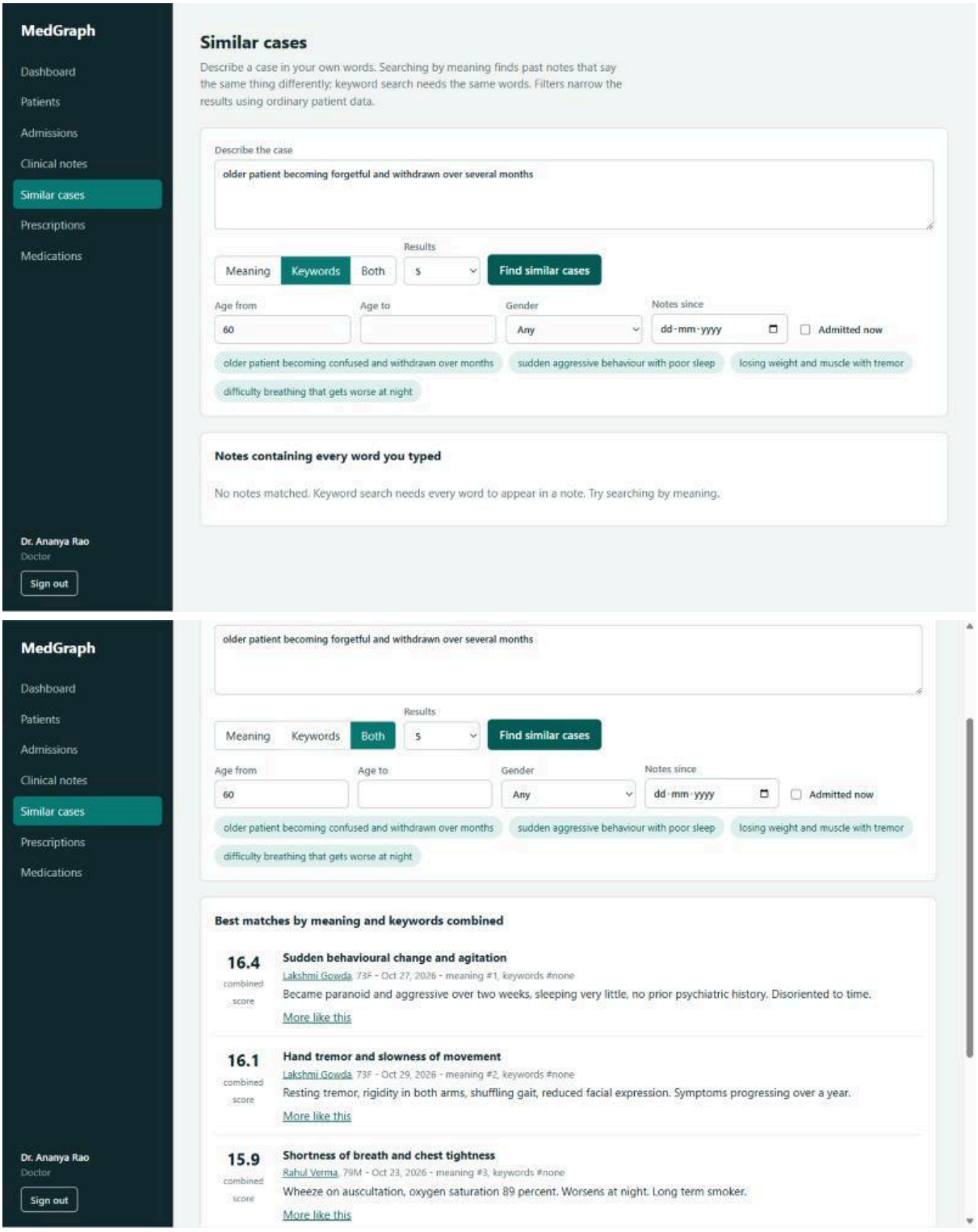


Screenshot 7: Similar case search by meaning

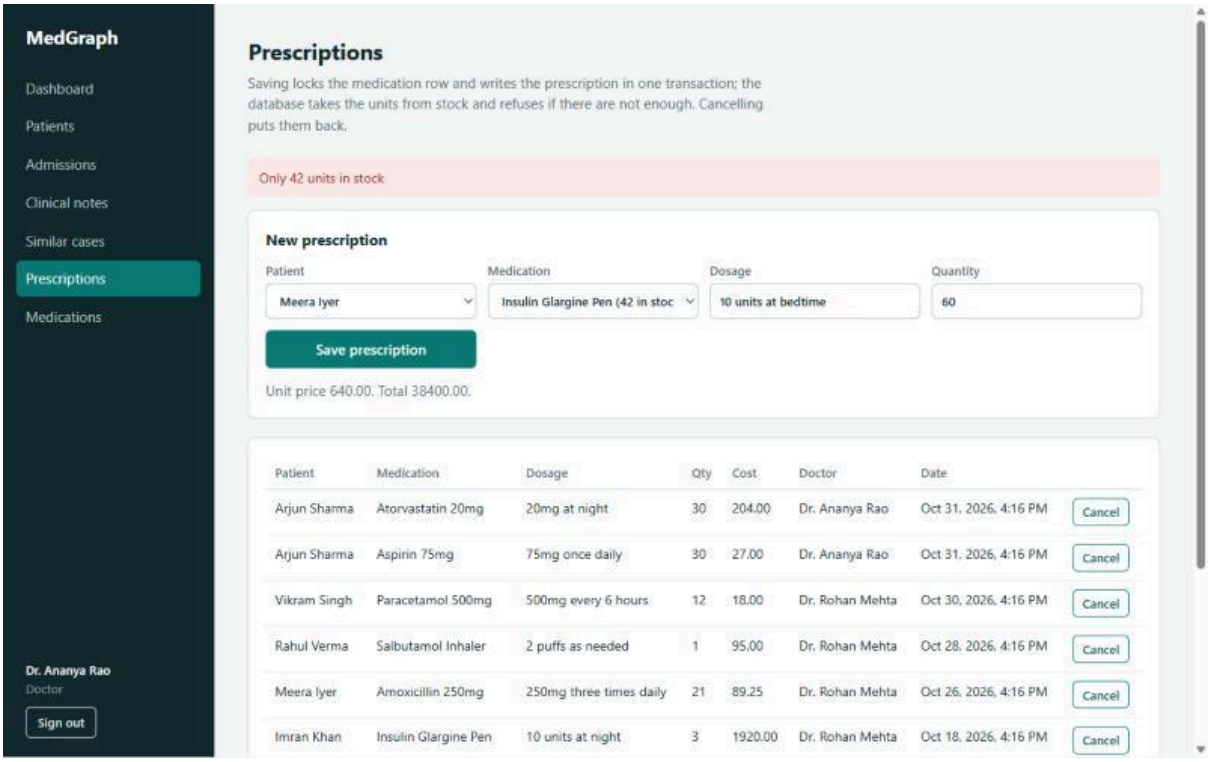




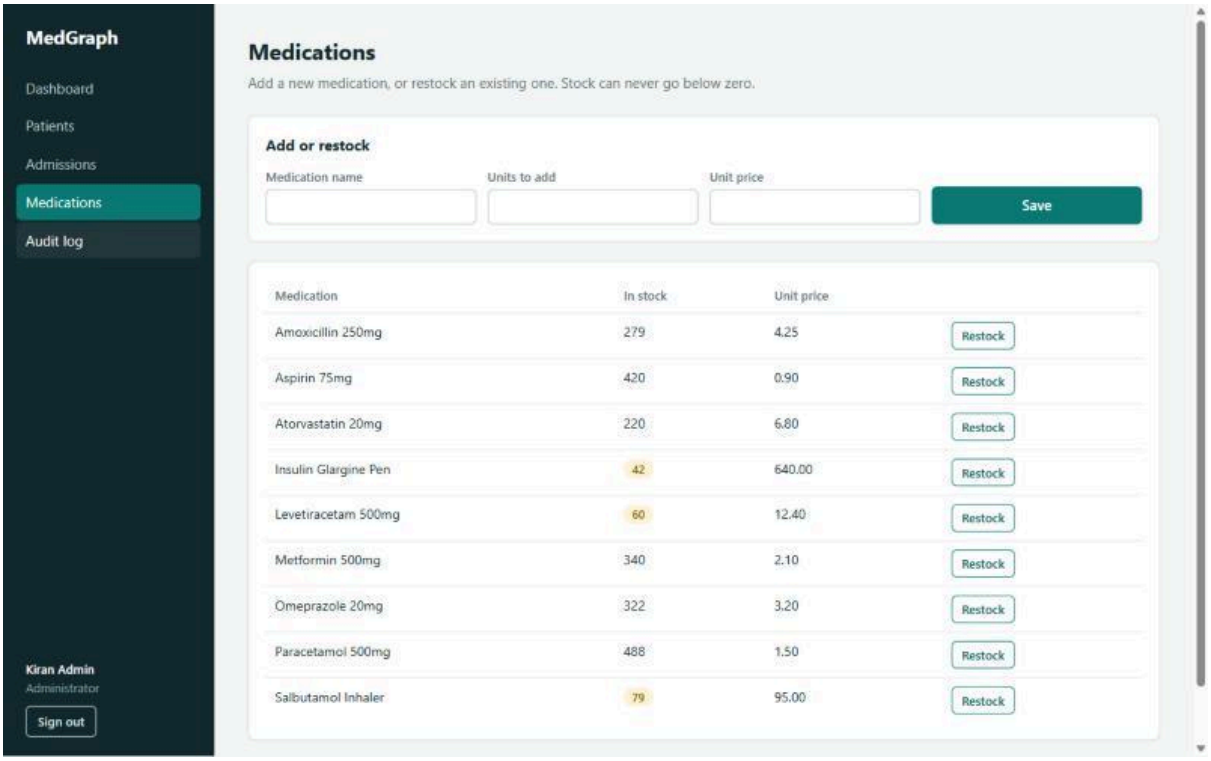
Screenshot 8: Similar case search by keywords, and with an age filter applied



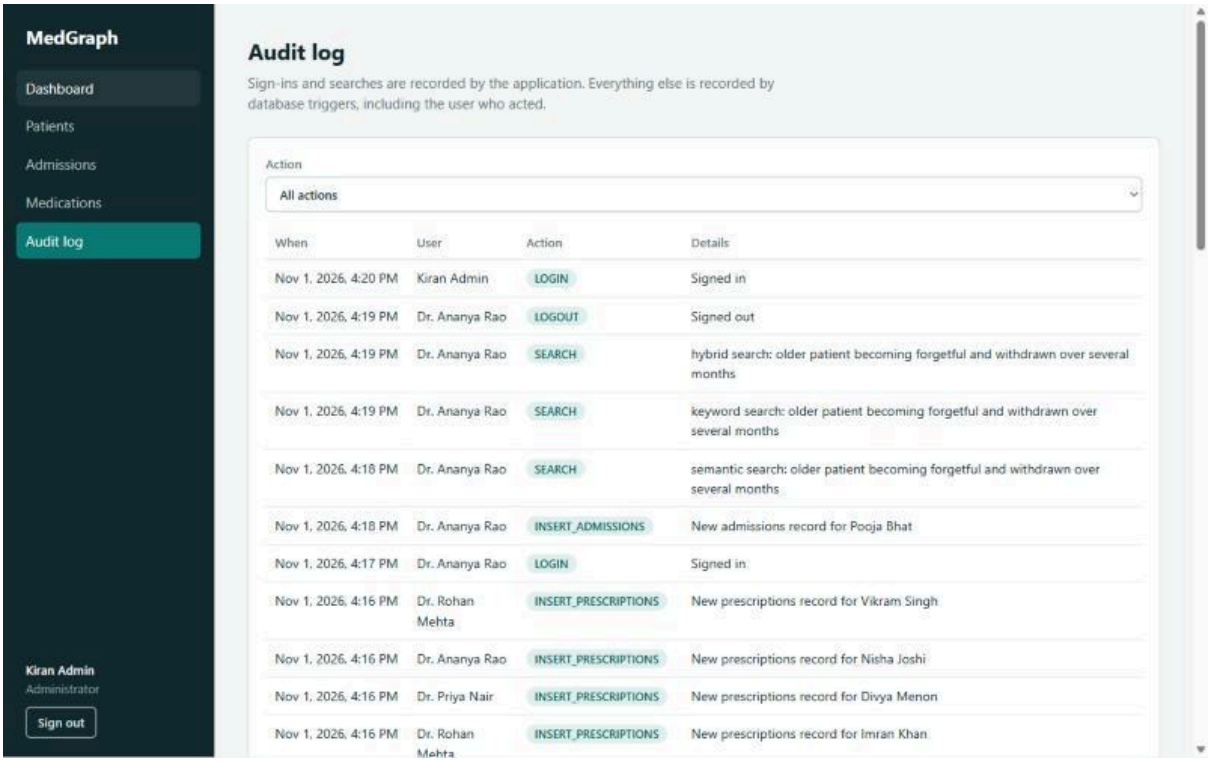
Screenshot 9: Prescriptions page, including the message shown when stock is too low



Screenshot 10: Medications page as admin



Screenshot 11: Audit log as admin



Screenshot 12: Code: database schema in the editor



```sql
-- MedGraph schema. PostgreSQL 16 + pgvector.
-- Constraints are named so the application can show a precise message for each one.
-- Setup order: 01_schema, 02_indexes, 03_functions_triggers, 04_views, 05_alter_drop_demo

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS users (
  user_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name     VARCHAR(100) NOT NULL CONSTRAINT chk_users_name CHECK (length(trim(full_name)) > 0),
  email         VARCHAR(100) NOT NULL CONSTRAINT uq_users_email UNIQUE
                CONSTRAINT chk_users_email CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
  password_hash VARCHAR(255) NOT NULL,                    -- bcrypt hash, never the password
  role          VARCHAR(10)  NOT NULL CONSTRAINT chk_users_role CHECK (role IN ('DOCTOR','ADMIN')),
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS patients (
  patient_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name        VARCHAR(50) NOT NULL CONSTRAINT chk_patients_first_name CHECK (length(trim(first_name)) > 0),
  last_name         VARCHAR(50) NOT NULL CONSTRAINT chk_patients_last_name  CHECK (length(trim(last_name)) > 0),
  dob               DATE        NOT NULL CONSTRAINT chk_patients_dob CHECK (dob > DATE '1900-01-01'),
  gender            CHAR(1)     NOT NULL CONSTRAINT chk_patients_gender CHECK (gender IN ('M','F','O')),
  blood_group       VARCHAR(3)  NOT NULL CONSTRAINT chk_patients_blood_group
                    CHECK (blood_group IN ('A+','A-','B+','B-','AB+','AB-','O+','O-')),
  phone_number      VARCHAR(20) NOT NULL CONSTRAINT chk_patients_phone CHECK (phone_number ~ '^[0-9+ -]{7,20}$'),
  emergency_contact VARCHAR(20) NOT NULL CONSTRAINT chk_patients_emergency CHECK (emergency_contact ~ '^[0-9+ -]{7,20}$'),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Age is not stored: it is derived from dob by patient_age() (a stored age would go stale).

CREATE TABLE IF NOT EXISTS wards (
  ward_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ward_name      VARCHAR(50) NOT NULL CONSTRAINT uq_wards_name UNIQUE,
  total_capacity INT NOT NULL CONSTRAINT chk_wards_capacity CHECK (total_capacity > 0),
  -- Deliberately redundant: equals the number of ADMITTED rows for the ward.
  -- Maintained by trigger trg_ward_occupancy, audited by view v_ward_consistency.
  occupied_beds  INT NOT NULL DEFAULT 0 CONSTRAINT chk_wards_occupied_nonneg CHECK (occupied_beds >= 0),
  CONSTRAINT chk_wards_occupied_within_capacity CHECK (occupied_beds <= total_capacity)
);

CREATE TABLE IF NOT EXISTS medications (
  med_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  med_name       VARCHAR(100) NOT NULL CONSTRAINT uq_medications_name UNIQUE,
  stock_quantity INT NOT NULL CONSTRAINT chk_medications_stock CHECK (stock_quantity >= 0),
  unit_price     NUMERIC(10,2) NOT NULL CONSTRAINT chk_medications_price CHECK (unit_price > 0)
);

CREATE TABLE IF NOT EXISTS admissions (
  admission_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id     UUID NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,
  ward_id        UUID REFERENCES wards(ward_id) ON DELETE SET NULL,
  doctor_id      UUID REFERENCES users(user_id) ON DELETE SET NULL,
  admission_date TIMESTAMPTZ NOT NULL DEFAULT now(),
  discharge_date TIMESTAMPTZ,
  status         VARCHAR(10) NOT NULL DEFAULT 'ADMITTED'
                 CONSTRAINT chk_admissions_status CHECK (status IN ('ADMITTED','DISCHARGED')),
  CONSTRAINT chk_admissions_discharge_order CHECK (discharge_date IS NULL OR discharge_date >= admission_date),
```

Screenshot 13: Code: triggers and stored procedure

```
-- ------------------------------------------------------------ pharmacy stock follows prescriptions
-- Locks the medication row first, so two prescriptions for the last units cannot both succeed.
CREATE OR REPLACE FUNCTION fn_decrement_stock() RETURNS trigger AS $$
DECLARE v_stock INT;
BEGIN
  SELECT stock_quantity INTO v_stock FROM medications WHERE med_id = NEW.med_id FOR UPDATE;
  IF v_stock IS NOT NULL AND v_stock < NEW.quantity THEN
    RAISE EXCEPTION 'Only % units in stock', v_stock USING ERRCODE = 'MG004';
  END IF;
  UPDATE medications SET stock_quantity = stock_quantity - NEW.quantity WHERE med_id = NEW.med_id;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_rx_stock ON prescriptions;
CREATE TRIGGER trg_rx_stock BEFORE INSERT ON prescriptions
  FOR EACH ROW EXECUTE FUNCTION fn_decrement_stock();
```

```
-- Locks BOTH wards in ward_id order, so two opposite transfers (A to B and B to A)
-- can never wait on each other. See scripts/concurrency-demo.mjs.
CREATE OR REPLACE PROCEDURE transfer_patient(p_admission UUID, p_new_ward UUID)
LANGUAGE plpgsql AS $$
DECLARE v_old UUID; v_cap INT; v_occ INT;
BEGIN
  SELECT ward_id INTO v_old FROM admissions
   WHERE admission_id = p_admission AND status = 'ADMITTED' FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'No active admission with that id' USING ERRCODE = 'MG003';
  END IF;
  IF v_old IS NOT DISTINCT FROM p_new_ward THEN
    RAISE EXCEPTION 'The patient is already in that ward' USING ERRCODE = 'MG005';
  END IF;

  PERFORM ward_id FROM wards WHERE ward_id IN (v_old, p_new_ward) ORDER BY ward_id FOR UPDATE;

  SELECT total_capacity, occupied_beds INTO v_cap, v_occ FROM wards WHERE ward_id = p_new_ward;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Ward not found' USING ERRCODE = 'MG003';
  END IF;
  IF v_occ >= v_cap THEN
    RAISE EXCEPTION 'That ward is full' USING ERRCODE = 'MG001';
  END IF;
  UPDATE admissions SET ward_id = p_new_ward WHERE admission_id = p_admission;
END $$;
```

Screenshot 14: Code: similar case search route (app/api/search/route.js)



```
JS route.js M ×
app > api > search > JS route.js > ...
1   import { NextResponse } from 'next/server';
2   import { query } from '@/lib/db';
3   import { authorize } from '@/lib/auth';import { dbError } from '@/lib/errors';
4   import { isUuid, intOrNull } from '@/lib/validate';import { embed, toVector } from '@/lib/embed.mjs';import { buildSearch, orQuery } from '@/lib/search';
5
6   // body: { query, mode: semantic | keyword | hybrid, k, like (note id), minAge, maxAge, gender, admittedOnly, since }
7   export async function POST(req) {
8     const { session, error } = await authorize(['DOCTOR']);
9     if (error) return error;
10    const b = await req.json().catch(() => ({}));
11
12    const mode = ['semantic', 'keyword', 'hybrid'].includes(b.mode) ? b.mode : 'semantic';
13    const k = Math.min(Math.max(intOrNull(b.k) ?? 5, 1), 20);
14    const text = String(b.query || '').trim().slice(0, 1000);
15    const likeId = b.like || null;
16    if (likeId && !isUuid(likeId)) return NextResponse.json({ error: 'Unknown note.' }, { status: 400 });
17    if (!likeId && !text) return NextResponse.json({ error: 'Describe the case you want to find.' }, { status: 400 });
18
19    const filters = {
20      minAge: intOrNull(b.minAge),
21      maxAge: intOrNull(b.maxAge),
22      gender: ['M', 'F', 'O'].includes(b.gender) ? b.gender : null,
23      admittedOnly: b.admittedOnly === true,
24      since: /^\d{4}-\d{2}-\d{2}$/.test(b.since || '') ? b.since : null,
25    };
26
27    try {
28      const needsVector = !likeId && mode !== 'keyword';
29      const vector = needsVector ? toVector(await embed(text)) : null;
30      const { sql, params } = buildSearch({ mode, vector, text: mode === 'hybrid' ? orQuery(text) : text, likeId, k, filters });
31      const { rows } = await query(sql, params);
32
33      let source = null;
34      if (likeId) {
35        source = (await query(
36          `SELECT n.chief_complaint, p.first_name || ' ' || p.last_name AS patient_name
37           FROM clinical_notes n JOIN patients p ON p.patient_id = n.patient_id WHERE n.note_id = $1`, [likeId])).rows[0] || null;
38      }
39      await query(`INSERT INTO audit_logs(user_id, action_type, description) VALUES ($1, 'SEARCH', $2)`, [
40        session.sub, likeId ? 'Searched for cases similar to a note' : `${mode} search: ${text.slice(0, 200)}`,
41      ]);
42      return NextResponse.json({ mode: likeId ? 'like' : mode, results: rows, source });
43    } catch (e) {
44      return dbError(e);
45    }
46  }
47
```

Screenshot 15: Output of npm run test:db

```
PS C:\Users\sakar\uni\sem5\dbms\mini proj\Medgraph> npm run test:db -- --model

> medgraph@1.0.0 test:db
> node scripts/test-db.mjs --model

T01  PASS  Occupied beds cannot exceed ward capacity
T02  PASS  Invalid blood group is rejected
T03  PASS  Malformed phone number is rejected
T04  PASS  Negative stock and zero price are rejected
T05  PASS  Duplicate email and invalid role are rejected
T06  PASS  Foreign key blocks a note for an unknown patient
T07  PASS  Discharge cannot precede admission; status must match dates
T08  PASS  Prescription quantity limit added later with ALTER TABLE
T09  PASS  Only DOCTOR accounts can be the responsible doctor
T10  PASS  Admit and discharge keep occupied_beds in step
T11  PASS  Admitting into a full ward fails with MG001
T12  PASS  A patient cannot have two active admissions
T13  PASS  transfer_patient moves the bed count from one ward to the other
T14  PASS  transfer_patient into a full ward fails and changes nothing
T15  PASS  Prescribing reduces stock; too large a quantity fails with MG004
T16  PASS  Deleting a prescription gives the stock back
T17  PASS  Audit rows record the acting user (app.user_id) for update and delete
T18  PASS  Deleting a patient cascades to notes, admissions and prescriptions
T19  PASS  Inserting a clinical note writes an audit row
T20  PASS  All views return rows and stored bed counts match reality
T21  PASS  B-tree, partial, GIN and HNSW indexes exist
T22  PASS  Full-text search with stemming and ranking
T23  PASS  Passwords are bcrypt hashes
T24  PASS  Every note has a 384-dimension embedding
T25  PASS  Every statement in sql/06_queries.sql runs
T26  PASS  Every statement in sql/07_dml_examples.sql runs
T27  PASS  Every statement in sql/09_explain.sql runs
T28  PASS  Semantic search ranks the memory-loss note first for a paraphrase
T29  PASS  Semantic search finds a note that keyword search misses

29/29 passed
PS C:\Users\sakar\uni\sem5\dbms\mini proj\Medgraph>
```

Screenshot 16: Output of npm run demo:concurrency

```
PS C:\Users\sakar\uni\sem5\dbms\mini proj\Medgraph> npm run demo:concurrency

> medgraph@1.0.0 demo:concurrency
> node scripts/concurrency-demo.mjs


=== 1. Lost update: two doctors each take 3 units from a stock of 10 (expected result: 4)
  without locking:    stock = 7   <- one update was lost
  SELECT ... FOR UPDATE: stock = 4   <- second transaction waited for the first

=== 2. The last bed: two admissions race for a ward with capacity 1 (admit_patient locks the ward row)
  session A: refused (MG001) That ward is full
  session B: admitted
  beds used: 1 of 1

=== 3. Deadlock: two transactions lock the same two wards in opposite order
  session A: committed
  session B: aborted by PostgreSQL (40P01): deadlock detected

=== 4. Same two locks, but both sessions lock in ward_id order (what transfer_patient does)
  session A: committed
  session B: committed (after waiting for A)

=== 5. Isolation levels: B changes the stock and commits while A is still inside a transaction
  READ COMMITTED  A reads 10, then 9   (non-repeatable read: A sees B's commit)
  REPEATABLE READ A reads 10, then 10   (repeatable: A sees its snapshot)
                  A then updates the row B changed: 40001 could not serialize access due to concurrent update  -> roll back and retry

Done. Temporary rows are being removed.
PS C:\Users\sakar\uni\sem5\dbms\mini proj\Medgraph>
```

9. Evaluation Rubrics - Completion Status

The status of each point in the project requirement sheet is given below.

Project title and real world problem: Completed

Team of two with defined responsibilities: Completed

Database requirement: PostgreSQL with a vector database (pgvector): Completed

Database design: ER diagram, relational schema, keys and constraints: Completed

SQL implementation: CREATE, ALTER, DROP, primary and foreign keys, indexes, views: Completed

Database operations: INSERT, UPDATE, DELETE, SELECT, joins, GROUP BY, HAVING, ORDER BY, subqueries: Completed

Advanced database features (four needed): transactions, views, functions, procedures, triggers, indexing, full text search, vector similarity: Completed

Vector database and semantic search: Completed

Web application interface in Next.js: Completed

Database and application integration through an API layer, with environment variables for configuration: Completed

Authentication and authorization with two roles (doctor and admin): Completed

Data validation and integrity at the database level: Completed

Security: hashed passwords, parameterised queries, no credentials in the source code: Completed

Project documentation (this report and the docs folder): Completed

Git and GitHub repository with README: Repository and README are ready, link and commit history to be shared

Testing results and demonstration: Completed. 29 database tests (27 plus 2 semantic search tests) passed; the concurrency demo shows the lost update, last-bed race, deadlock and isolation level cases. Output in Screenshots 15 and 16 and in docs/results/.